Machine Learning

Created time	July 23, 2026 9:24 PM
Last edited time	July 27, 2026 8:39 PM
Tag	Year 4 Term 2

Resources:

- Ma'am's slides
- CS229 Lecture Notes ~ Andrew Ng
- Understanding Feature Vectors and Spaces | PDF | Support Vector Machine | Machine Learning
- CSE471.pdf - Google Drive - Mahir Labib Dihan Sir
- Machine Learning textbook ~ Tom Mitchell, McGraw Hill, 1997.
- DATA CLUSTERING: Algorithms and Applications
- Machine Learning Videos - My fav

Introduction

Machine learning is a field of study that gives computers the **ability to learn without being explicitly programmed.**

Aspect	Machine Learning	Traditional Programming
Approach	Learns patterns from data	Explicit rules written by programmer
Input	Data + desired outputs (labels)	Rules/logic + input data
Output	Model that predicts or classifies	Deterministic results based on rules
Flexibility	Adapts to new/unseen data	Limited to predefined rules
Error Handling	Accepts some error; focuses on accuracy	Must handle all cases explicitly

Aspect	Machine Learning	Traditional Programming
Development Effort	Requires large datasets and training	Requires detailed rule design and coding
Examples	Spam detection, image recognition, speech-to-text	Calculator, payroll system, sorting algorithm
Performance Dependency	Quality and quantity of data	Quality of programmer's logic
Nature of Solution	Probabilistic (approximate answers)	Deterministic (exact answers)

Attributes/Features/Properties: An attribute is a measurable characteristics/heuristic property of a a phenomenon being observed.

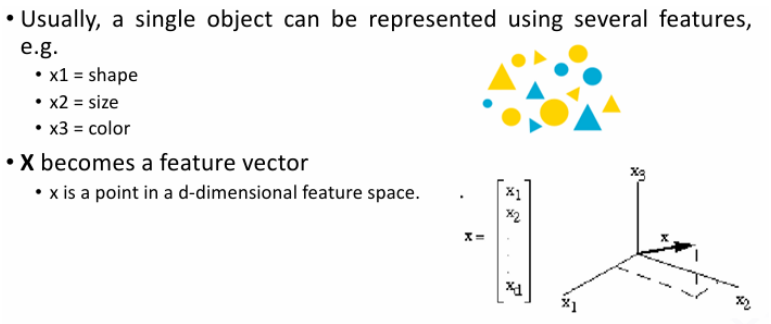
- **Heuristic nature:** Attributes are chosen because they are **practical and discriminative ,** they help distinguish between different classes or outcomes.

It is represented by **feature vector**.

A feature vector is an n**-dimensional vector** of **numerical values** where each dimension corresponds to one **features** (attributes)

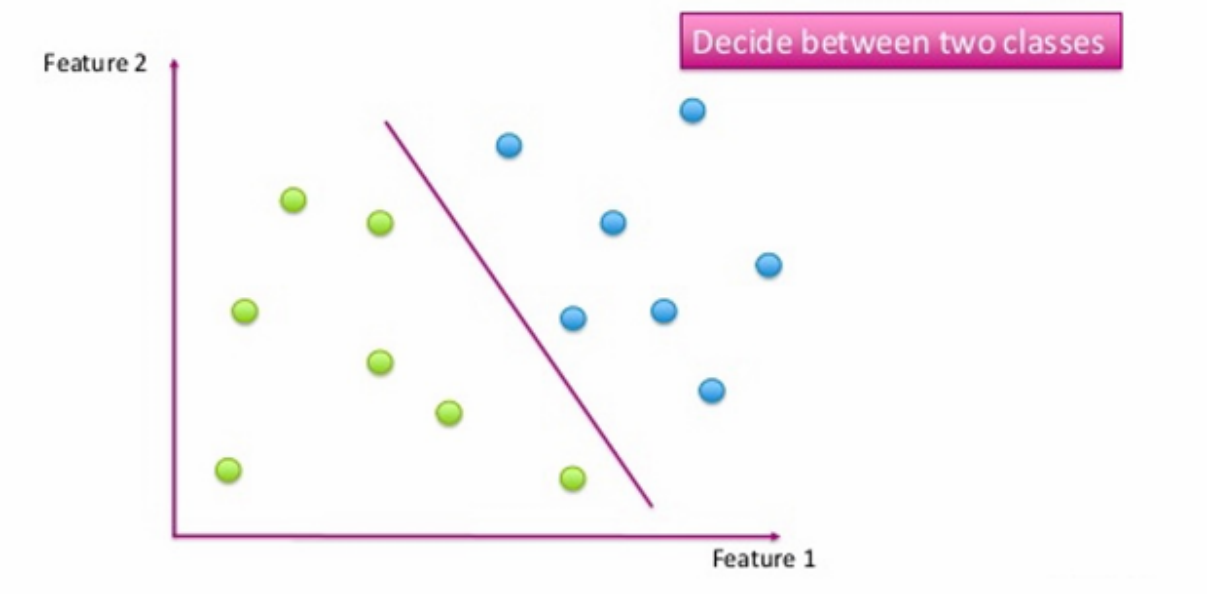
- It converts real-world objects into a mathematical form suitable for algorithms.
- Feature vectors are the inputs to machine learning models.

Example:

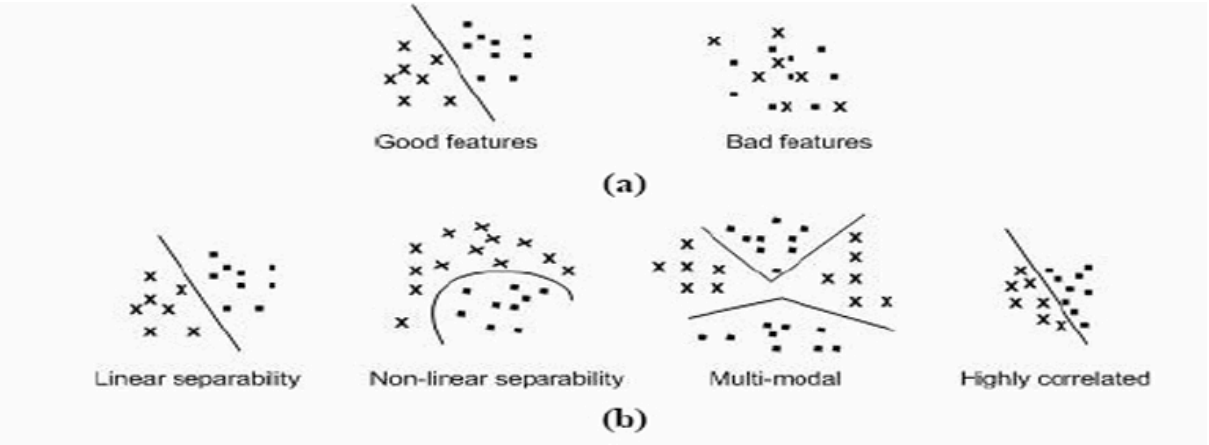


A feature space is a **conceptual space** where each **data point** is represented by a set of features or attributes.

Example:



A decision boundary is the **surface** that **separates different predicted classes i**n the feature space. Borders between decision regions are called decision boundaries



The goal of a classifier is to partition the feature space into class-labeled decision
regions.

In **machine learning**, the **target** refers to the **output variable** (also called the dependent variable, **label**, or **response**) that the model is trying to predict.
Example: Email is "Spam" or "Not Spam."

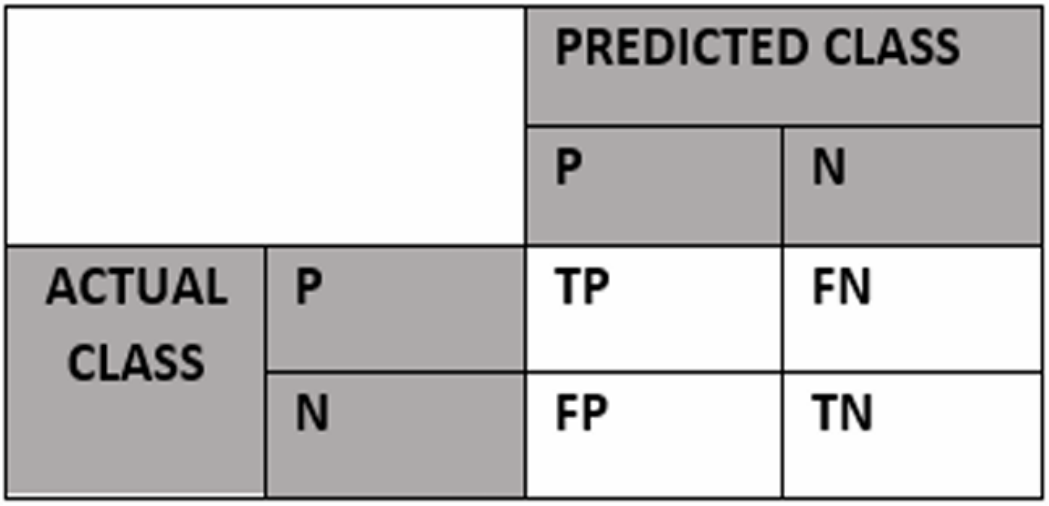
Evaluation of Classification and Regression Models

Model evaluation is the process that uses some metrics which help us to analyze the performance of the model.

Metrics for Performance Evaluation: It refers to the **quantitative and qualitative measures** used to asses **how well** a machine learning **model perform.**

Metrics for Performance Evaluation for classification

1. **Confusion Matrix:** Confusion Matrix is a performance evaluation method used to **assess classification** models by **comparing predicted labels with actual labels**.



- A confusion matrix is an $N \times N$ matrix where N is the number of target classes.
  It represents the number of actual outputs and the predicted outputs.
- True Positives: It is also known as TP. It is the output in which the actual and the predicted values are YES.
- True Negatives: It is also known as TN. It is the output in which the actual and the predicted values are NO.
- False Positives: It is also known as FP. It is the output in which the actual value is NO but the predicted value is YES.
- False Negatives: It is also known as FN. It is the output in which the actual value is YES but the predicted value is NO.

2. **Accuracy:** Accuracy is defined as the ration of the number of correct prediction to the total number of predictions.

   $$\text{Accuracy} = \frac{\text{TP}+\text{FN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}$$

   - Most fundamental metrics
   - Drawback: Cannot perform well on an imbalanced dataset

3. **Precision:** Precision is the ratio of true positive to the the summation of true positives and false positives.

   $$\text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FN}}$$

   - Analyze the positive prediction
   - Drawback: Doesn't consider True Negatives and False Negatives.

4. **Recall:** Recall is the ratio of the true positives to the summation of true positives and false negatives.

   $$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}$$

   - Analyze the number of correct positive samples
   - Drawback: It often leads to a **higher false positive rate**

5. **F1-measures:** The F1 score is the **harmonic mean** of precision and recall.

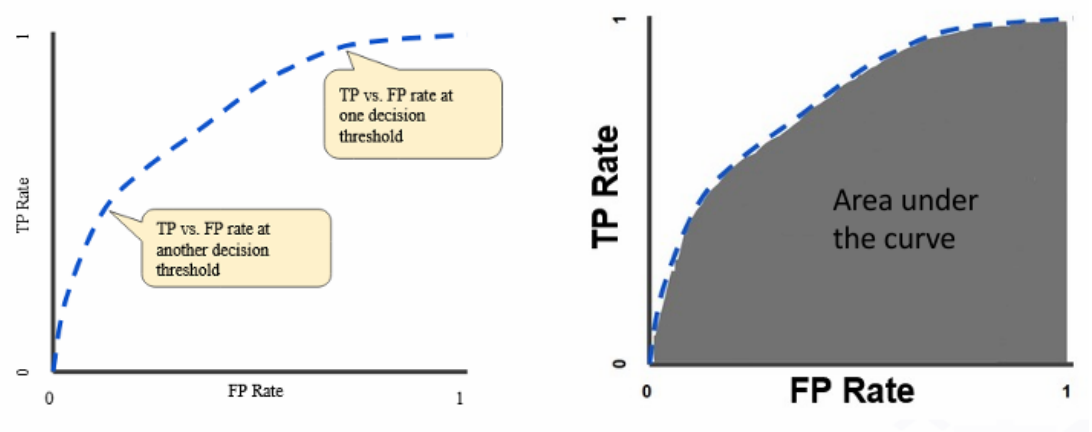
   $$\text{F1} = \frac{2\times\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$$

   - Precision-recall tradeoff : One increase, another decreases or vice versa
   - F1 Score is to combine precision and recall

6. **AUC-ROC Curve:** AUC (Area Under Curve) is an evaluation metric that is used to analyze the classification model at different threshold values. The Receiver Operating Characteristics (ROC) curve is a probabilistic curve used to highlight the model's performance. The curve has two parameters:

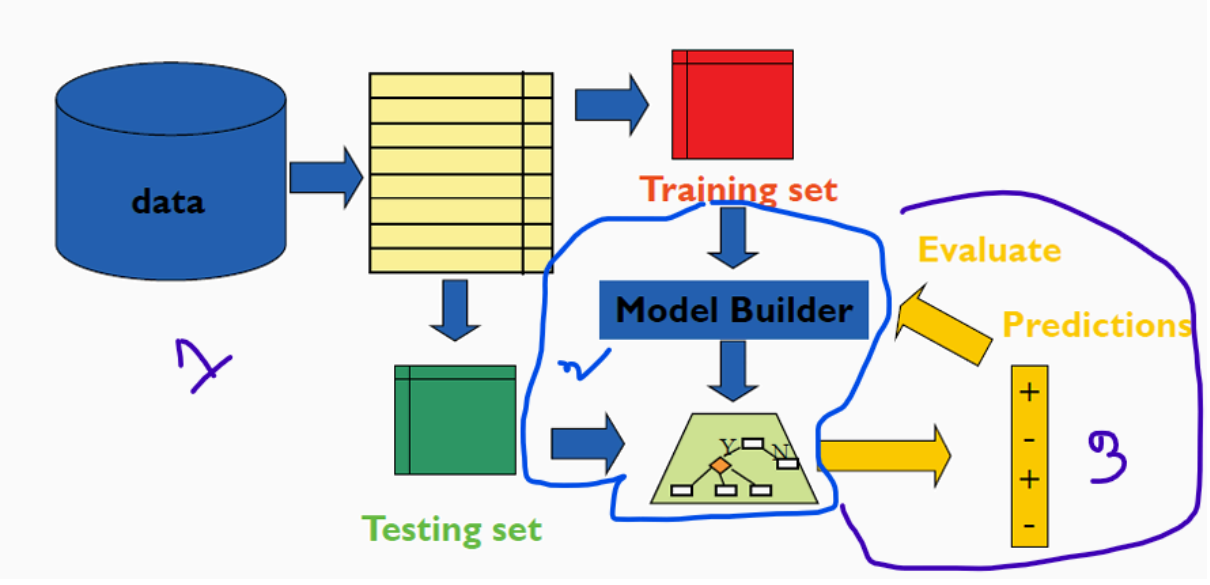
   - TPR: It stands for True positive rate. It basically follows the formula of Recall.
   - FPR: It stands for False Positive rate. It is defined as the ratio of False positives to the summation of false positives and True negatives.

     $$\text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}}$$

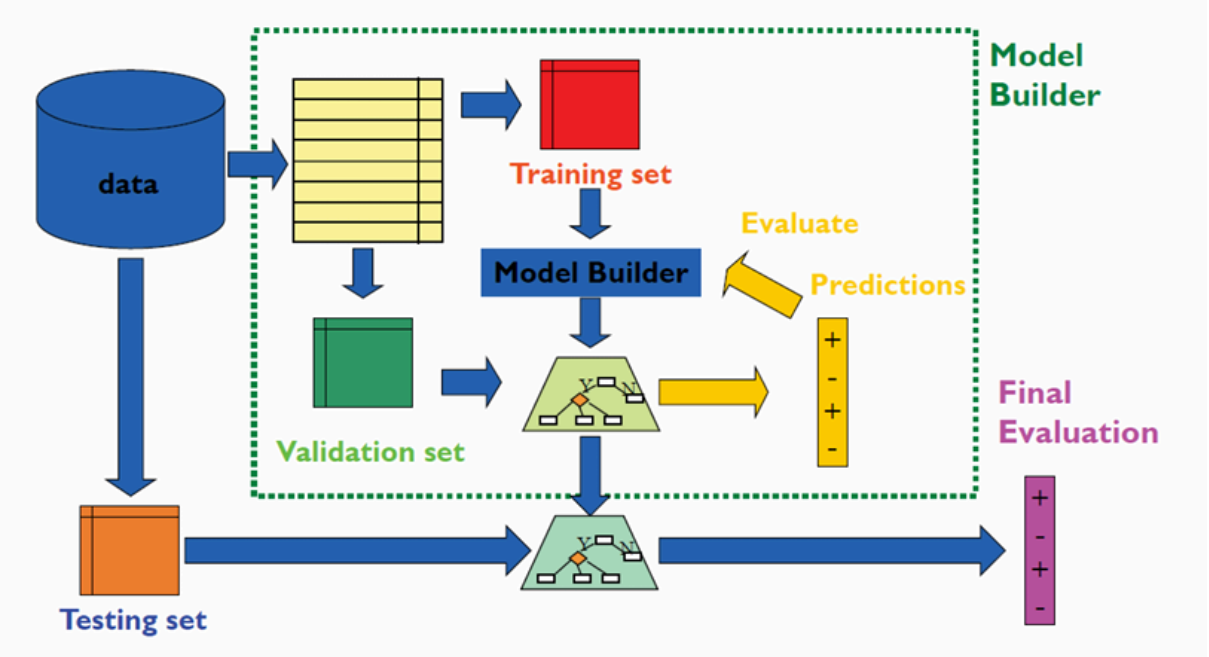


Basic workflow for performance evaluation in classification task

1. **Split the data between train and test set:** Divide the available data into a training set and a test set, ensuring both represent the overall distribution.
2. **Compute a model using the data in the training set:** Use the training set to build and fit the classification model, learning patterns from the data.
3. **Evaluate the model using the test set:** Test the trained model on the test set to measure its performance using metrics like accuracy, precision, recall, and F1-score.



- One evaluation is complete, all the data can be used to build the final classifer.
- Generally the larger teh training data the better the classifer.



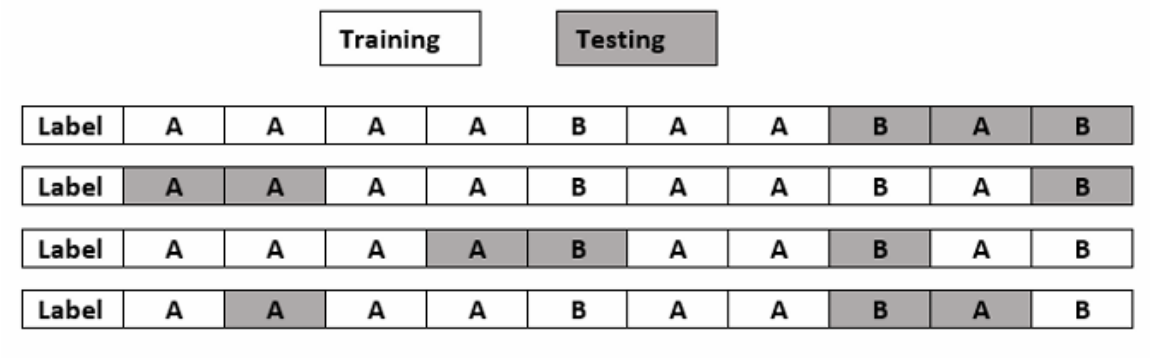
The Full Process: Training, Validation, and Testing

Aspect	Training Set	Validation Set	Testing Set
Used for	Learning patterns	Tuning hyperparameters	Final evaluation
When used	During training	During training (but separate from training data)	After training is complete
Risk if misused	Overfitting if too small	Data leakage if mixed with test set	Inflated results if used too early

Methods of Estimation: Refers to the strategies used to estimate how well a model will perform on unseen data. These methods help us **avoid overfitting** and give a **realistic measure of generalization.**

1. **Holdout:** The holdout method is a simple technique for estimating the performance of a machine learning model by splitting the dataset into two parts: training and testing.
    a. Reserves a certain amount for testing and uses the remainder for training
    b. Performance depends heavily on the way the data is split.
    c. May give biased or unstable results if the dataset is small.

2. **Repeated Holdout:** Repeated holdout improves **reliability** by performing the holdout process multiple times with **different random splits** of the dataset.
   a. In each iteration, a certain proportion is **randomly selected for training**
   b. The error rates on the different iterations **are averaged to yield an overall error rate**
   c. Provides a more trustworthy estimate of generalization.
   d. More computationally expensive than a single holdout

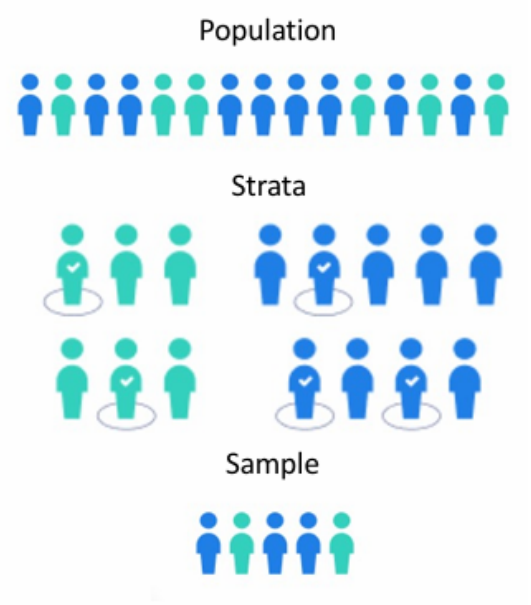


3. **Stratified Sampling:** Stratified sampling is a method of splitting data into training and testing sets (or folds) while **preserving the proportion of classes**. It ensures that each **subset has the same class distribution** as the original dataset.

   Steps:

   a. **Identify the classes** in the dataset (e.g., Positive vs. Negative).
   b. **Divide the dataset into strata** (groups) based on class labels.
   c. **Sample proportionally** from each stratum when creating training and testing sets. (Makes sure that each class is represented with approximately equal proportions in both subsets)
   d. **Use these stratified sets** for training, validation, or cross-validation.

In many classification datasets, one class (majority) has far more samples than the other (minority). But if the dataset itself is highly imbalanced, stratification alone won't fix the problem , the imbalance will persist in both train and test sets.



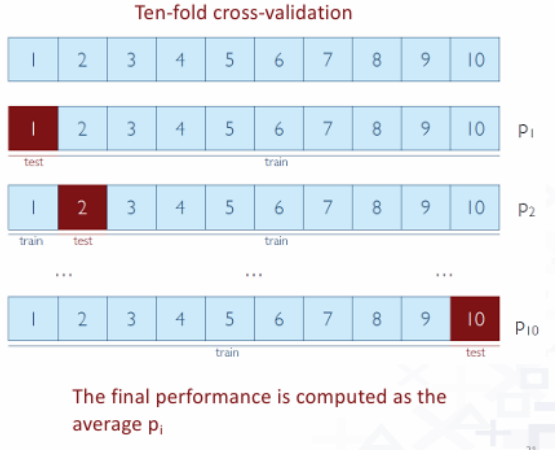
- Oversampling method **increases** the **minority** class samples to reduce the degree of uneven distribution.
- Undersampling **removes** the sample from the **majority** class.

4. **Bootstrapping:** Bootstrapping is a **resampling** method used to estimate the performance of a model by **creating multiple training and testing sets** from the original dataset through **sampling with replacement.**
    - **Original dataset:** Start with your full dataset of size *N*.
    - **Sampling with replacement:** Randomly select *N* samples from the dataset, allowing duplicates.
    - Works well when the dataset is **small**.

- Pick one number.
- Write it down.
- **Put it back** before picking again.

5. **Cross-validation:** Cross-validation is a technique in machine learning used to **assess how well a model generalizes** to unseen data. Instead of relying on a single train/test split, it systematically splits the dataset into multiple parts (folds) and evaluates the model across them.

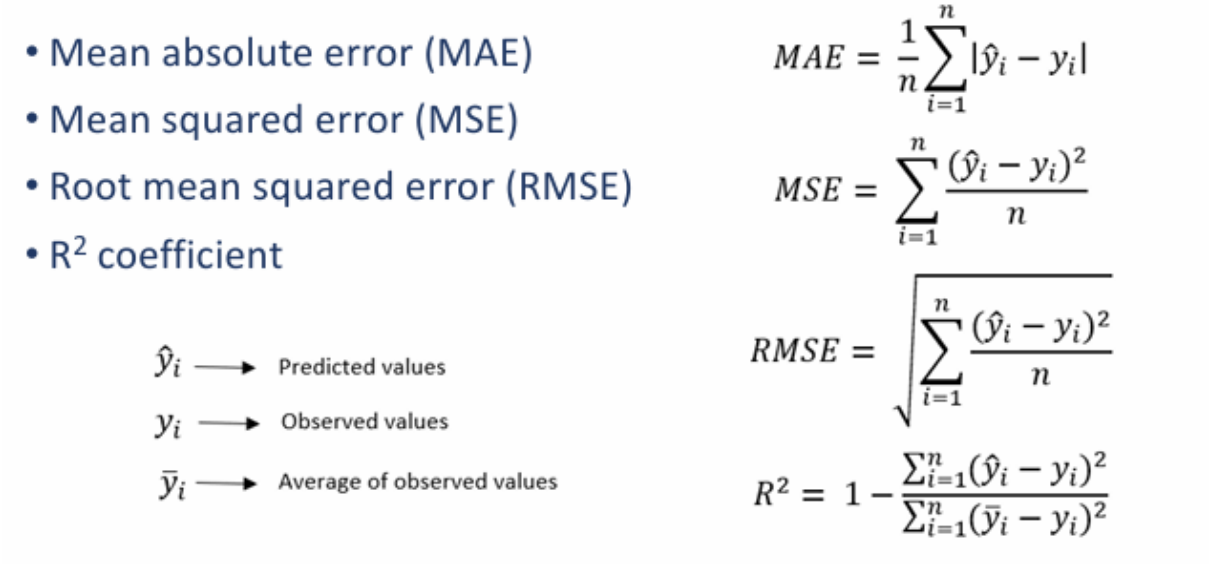
- Data Split into k subsets of equal size
- Each subset in turn is used for testing and remainder for training.
  - Train the model on $k-1$ fold
  - Test the model on the remaining fold
  - Repeat until each fold has served as the test set once
- This is called k-fold cross validation and overlapping test sets
- The error estimates are averaged to yield an overall error estimate



Standard method for evaluation stratified ten-fold cross-validation. Because, it is the best choice.

6. **Leave-One-Out Cross-Validation:** Leave-One-Out-Cross-Validation is a special case of cross-validation where the number of folds k euqals the number of samples N. Each single data point is used once as a test set, while the remaing $N-1$ samples are used for training.
   - Take the dataset of size N
   - Use $N-1$ samples for training and leave 1 sample out for testing.
   - Repeat the process, leave out a different sample each time.
   - Continue until every sample has been used once as the test set.
   - Average the performance.
   - Computationally expensive, makes best use of the data, involves no random subsampling.

Performance Evaluation Metrics (Regression)



Classification

Definition Classification Data Mining: Given a training data set $D = \{t_{1,2}, ..t_n\}$ which has n rows (records, tuples) and m columns (features, attributes) for a given target (selected feature, output) concept with classes $C = \{c_1, c_2, \ldots c_p\}$ discover by a model for predicting a target value c_i of a new instance.

Classification application development : A three-phase process

Model construction: find classification rules from the training data set

1. **Training:**
    a. Each tuple is assumed to belong to a predefined class, as determined by the class **label attribute.**
    b. The set of tuples used for model construction is training set
    c. The model is represented as *classification rules, decision trees, or distrubuted network weights.*

Model Usage: for classifying unseen objects

2. **Testing:** Estimate accuracy of the model

    a. The known label of **test sample is compared with the classified result** from the model
    b. **Accuracy rate** is the percentage of test set samples that are correctly classified by the model
    c. Test set is i**ndependent of training set**
3. **Predicting:** If the accuracy is acceptable, use the model to classify data.

Evaluating classification method

1. **Predictive accuracy**
2. **Speed and scalability**
    a. Time to construct and use the model
    b. Ability of handling large data set
3. **Robustness**
    a. Handling noise and missing values
4. **Interpretability**
    a. Understanding and insight provided by the model
5. **Goodness of rules**
    a. decision tree size
    b. compactness of classification rules

Data preprocessing for classification

1. **Data cleaning:** Preprocess data in order to reduce noise and handle missing values
2. **Relevance analysis (feature selection):** Remove the irrelevant or redundant attributes Keep only the most useful attributes for modeling.
3. **Data transformation:** Convert data into a format suitable for analysis.
    a. Generalize or normalize data
    b. Use discretization techniques to convert a given continuous attribute into categorical attribute

c. Equal width partitioning method

Decision Tree Learning

Decision tree learning is a method for approximating discrete-valued target functions, in which the learned function represented by a decision tree. Learned tree can also be represented as sets of if-then rules to improve human readability. It is one of the most popular **inductive inference algorithm.**

Induction

It is a **reasoning approach** where a concept **can be learned/supported by gathering evidences** from observation.

Decision Tree

A DT is decision support tool that uses a **graph of tree or model of decisions** and their possible consequences, including **chance event outcomes,** resource **costs and utility.** An **internal node** of DT associates with a **selected attribute and arcs** with values for that attribute.

Define DT

Given:

- Training data set D with n tuples: $D = \{t_1, .., t_n\}$
- Schema of D with m attributes : $(A_1, A_2, .., A_m)$
- Classes: $C = \{C_1, \ldots, C_p\}$ of a selected target concept

Decision Tree is defined

- A root node has no incoming arcs and zero or more outgoing arcs
- Each internal node is a selected attribute, A_i
- Each arc is labeled with an attribute value of the parent node
- Each leaf node is labelled with a class, C_j

How to draw DT : ID3 Algorithm YT: How to draw DT by ID3

Our basic algorithm, **ID3 (Iterative Dichotomiser 3),** learns decision trees by constructing them top-down, beginning with the question, “which attribute should be tested by the root of the tree?”

The central choice in the ID3 algorithm is selecting which attribute to test at each node in the tree. We would like to select the attribute that is most useful for classifying examples. What is a good quantitative measure of the worth of an attribute? We will define a statistical property, called information gain, that measures how well a given attribute separates the training examples according to their target classification. ID3 uses this information gain measure to select among the candidate attributes at each step while growing the tree.

Information theory

Entropy: Entropy characterizes the impurity of an arbitrary set of data. For classification, entropy is used to estimate how close a data set to a single class.

$$\text{Entropy, S } = \sum -p_i \log_2 p_i$$

$$\text{Gini Index} = 1 - \sum p_i^2$$

$$\text{Classification Error} = 1 - \max\{p_i\}$$

K-Nearest Neighbors (KNN)

KNN is a simple supervised ML algorithm used for classification and regression. It predicts the label or value of a new data point by finding the K closet points in the training dataset and using majority voting or averaging.

- KNN is a memory-based classifier
    - It doesn’t build a model but relies directly on the training data

❓ A hospital has a database of previous patients containing their:

- Blood pressure
- Glucose level
- BMI
- Diagnosis (Diabetic or Non-Diabetic)

When a new patient arrives, the hospital wants to classify the patient by comparing them with the most similar previous patients based on these measurements. The hospital does not want to build an explicit predictive model and prefers to use the stored historical cases directly.
Question: Which machine learning algorithm would be most suitable for this application? Justify your answer.
Answer:
Algorithm: k-Nearest Neighbors (k-NN)
Justification:

- The hospital wants to classify a new patient by comparing them with the **most similar previous patients**.
- It does **not** want to build an explicit **predictive model.**
- k-NN stores all historical data and classifies a new instance **based on the majority class of its nearest neighbors** using similarity (distance) measures.

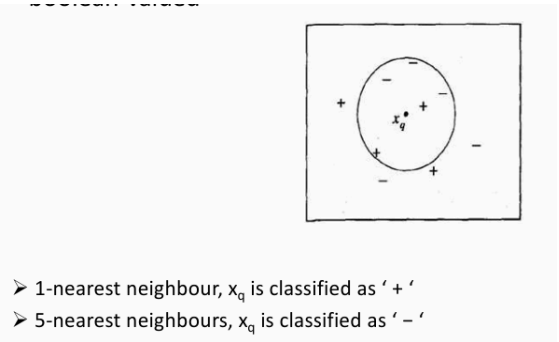
- **How does it work**
  - **Determine parameter** k, the number of nearest neighbors
  - **Calculate the distances** between the query instance and all the training samples by using the **standard Euclidean Distance**

    $\text{dist}(x, x') = \sqrt{\sum_{s=1}^{n}(x_s - x'_s)^2}$
    - $x =$ the first data point (query/test)
    - $x' =$ the second data point (a training instance)
    - $x_s =$ the value of the s-th feature of the query instance

        - $x'_s =$ the value of the s-th feature of the training instance
        - $n =$ the total number of features
    - **Determine the k nearest neighbors** based on the distances and gather the class labels of the k nearest neighbors
        - For discrete-values target function, assign the label with **majority** in the k nearest neighbors as the predicted class label of the query instance
        - For continuous-value target function, compute the **mean value** of the k nearest trainng examples.

Drawbacks

1. The fixed value of k
2. All neighbours have an equal contribution to the predict a target class
3. Choice of metric
4. Difficulty of handling unbalanced data

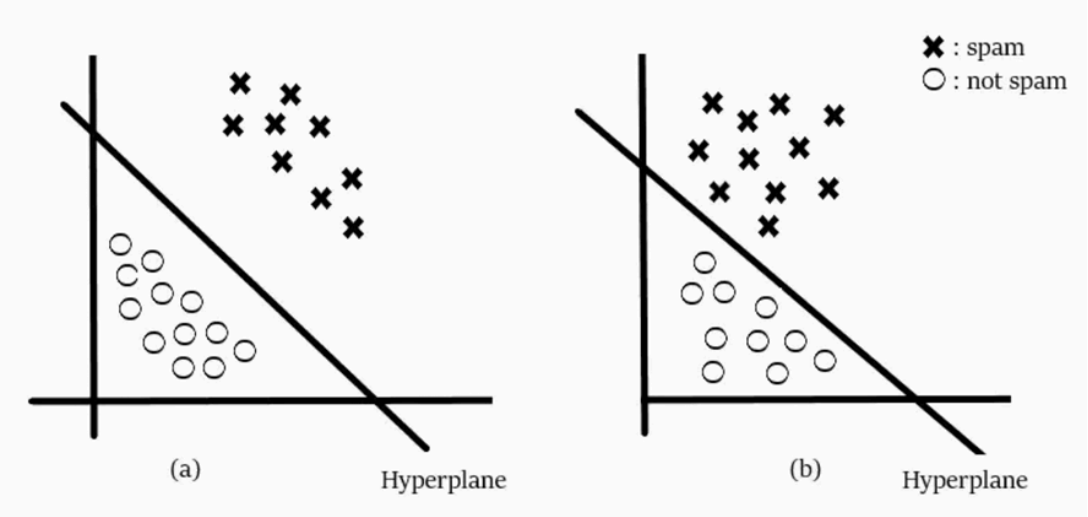


SVM : Support Vector Machine SVM on YT

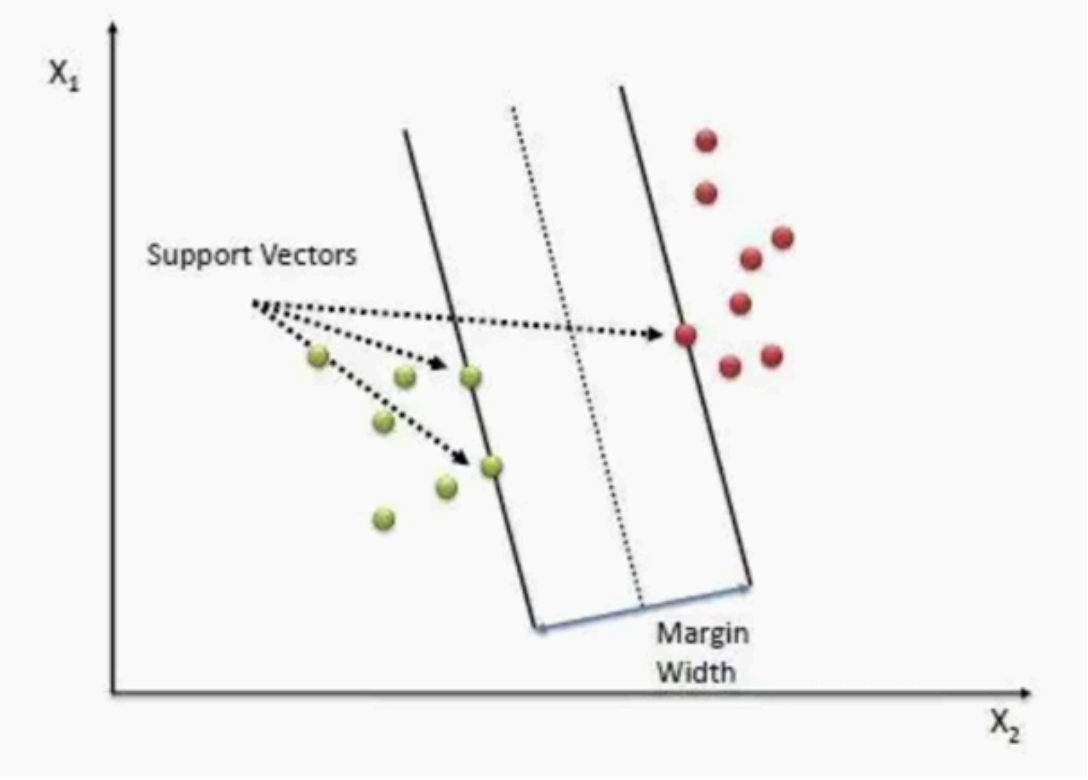
SVM are supervised machine learning used for classification and regression tasks.

- efficient for linear and non-linear classification

- **Hyperplane :** A hyperplane is a decision boundary that **separates a points of different classes** in the feature space.
    - Mathematical representation : $w \cdot x + b = 0$



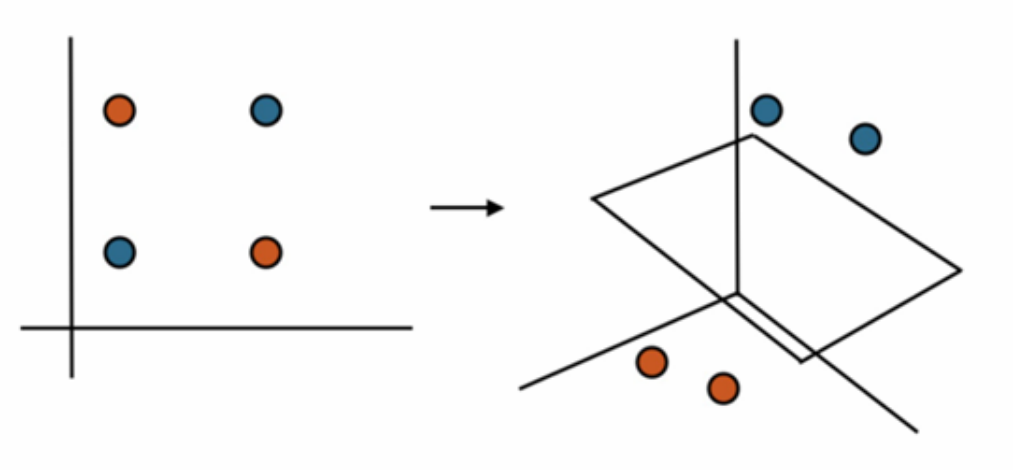
- **Support vector:** These are the data points that are **closet to the hyperplane**. These points influence the **orientation and position** of the hyperplane.



- **Margin :** The margin is the **distance** between **hyperplane and the nearest data point of either class**. SVM tries to **maximize the margin improve classification** confidence.
    - **Hard Margin:** Data is **perfectly separable** by the hyperplane.

- **Soft Margin:** Allow some **misclassification or margin violations** for non linearly separable data.

- **Kernel Function:** A kernel function **transforms** data into a **higher-dimensional** space to make it linearly separable.



  - **Linear Kernel :** Suitable for linearly separable data.
  - **Polynomial Kernel:** For curved boundaries.
  - **Radial Basis Function (RBF) Kernel:** Captures complex relationship.

  SVM's are very robust to noise in the data and can avoid the loss of generalization if the parameters for a given kernel function are chosen appropriately. However, there is not a universal selection criterion for choosing which function can be used to best classify data.

Regression: Regression is a fundamental concept in statistics and machine learning that focuses on **modeling the relationship** between input variables (features) and an output variable (target).

Linear Regression

Linear regression is a predictive modelling technique that estimates the relationship between a dependent variable y and one or more independent variables $x_1, x_2, \ldots, x_n$ by fitting a linear equation to observed data.

$Y = mx + b$, m and b are regression coefficient.

Formula to calculate slope and intercept:

$m = \frac{\sum(x-\bar{x})\sum(y-\bar{y})}{\sum(x-\bar{x})^2}; b = \bar{y} - m\bar{x}$

❓ Make prediction for study hours = 6

(x)	(y)
1	2
2	4
3	6
4	8
5	10

$\bar{x} = \frac{1+2+3+4+5}{5} = 3, \bar{y} = \frac{2+4+6+8+10}{5} = 6$

x	y	$(x - \bar{x})$	$(y - \bar{y})$
1	2	-2	-4
2	4	-1	-2
3	6	0	0
4	8	1	2
5	10	2	4

$m = \frac{20}{10} = 2$
$b = 6 - 3 \cdot 2 = 0$
$y = mx + b = 2x$
$x = 6 \Rightarrow y = 2 \times 6 = 12$

Logistic Regression

- Logistic regression models the **probabilities of the different possible outcomes** of a **response variable**, given a set of input variables. The model is appropriate for two-class classification or prediction problem.
- The logistic regression classifier can be derived by analogy to the linear regression hypothesis

$Y = \text{logistic}(mx + c)$

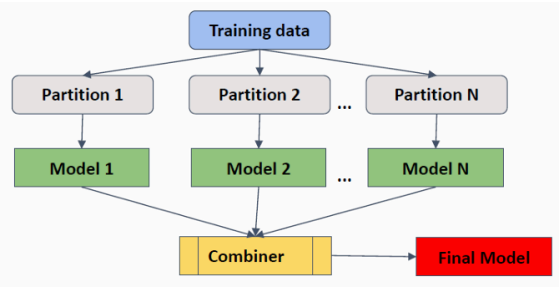
$\text{logistic} = \text{Sigmoid function}, \sigma(z) = \frac{1}{1+e^{-z}}$

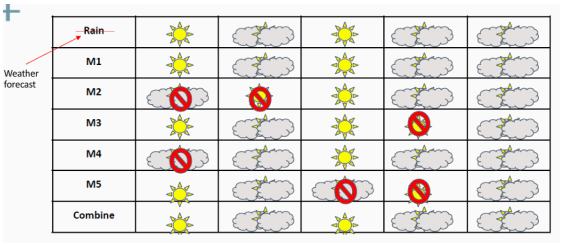
Ensemble Learning

It uses multiple learning algorithm to obtain better predictive performance.

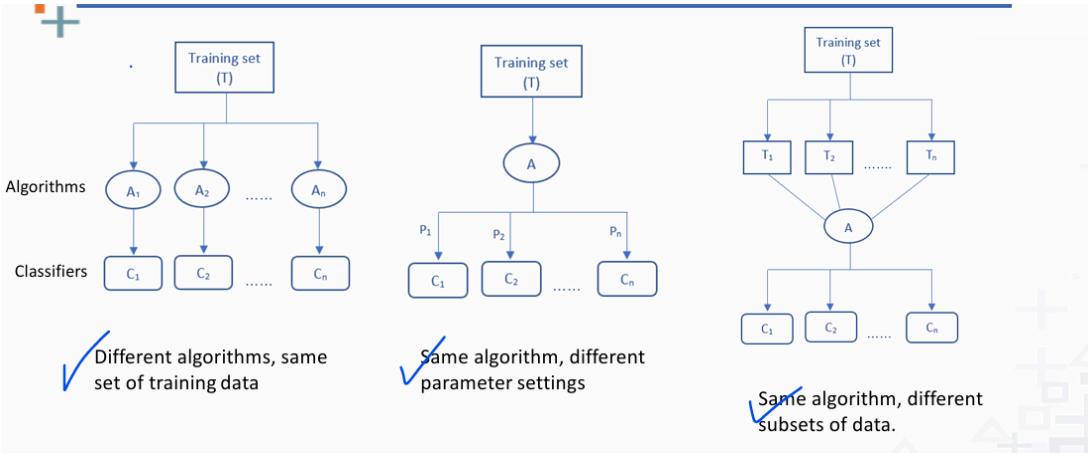
No Free Lunch Theorem : There is no algorithm that is always the most accurate.

- Learn multiple alternative models using different training data or different learning algorithm.
- Combines decisions of multiple models.
- Less bias and less variance





Types of Ensemble Learning

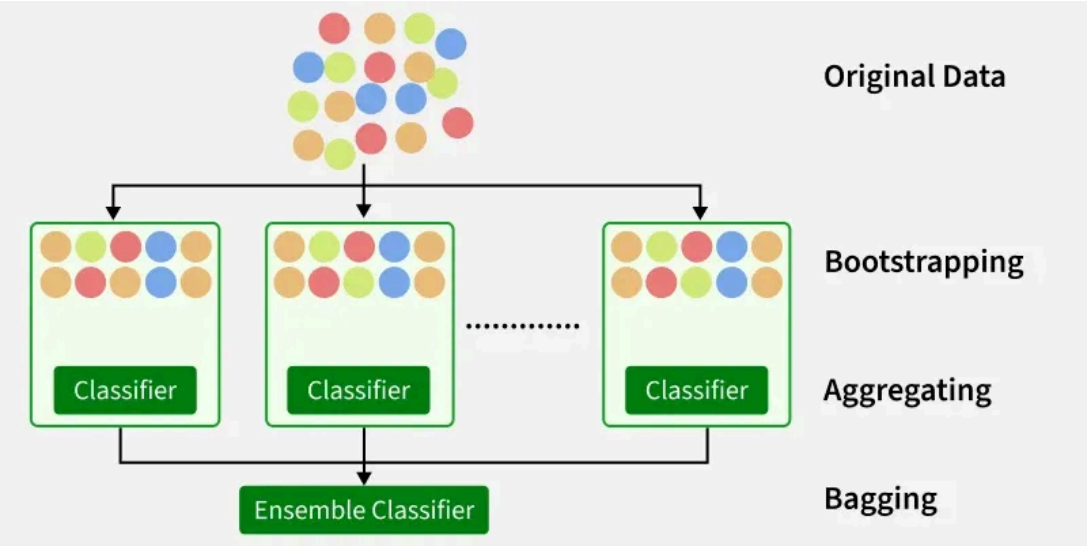


Bagging

Bagging or Bootstrap Aggregating works by training multiple base models independently and in parallel on different random subset of the training data.

- Improve accuracy
- Reduce Overfitting

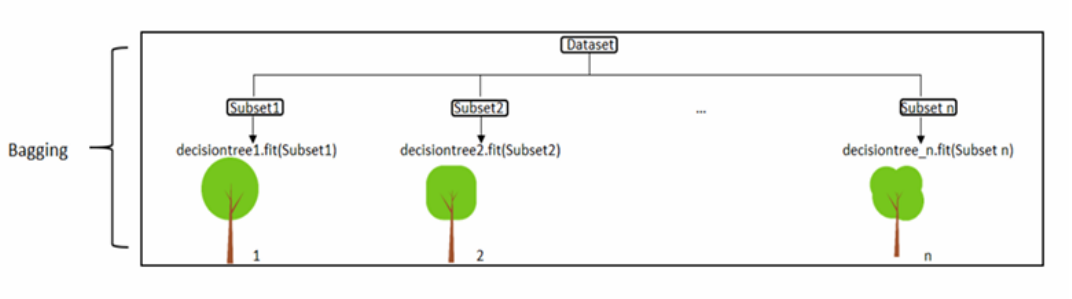
- Lower Variance



Working

- **Bootstrap Sampling :** From the original dataset, multiple training subset are created by sampling with replacement.
  - Reduce overfitting

- **Base Model Training:** Each bootstrap sample trains an independent base learners.
  - Training happens in parallel
- **Aggregating:** Once trained, each base model generates predictions on new data. For classification, predictions are combined via majority voting. Mean for regression.



- **Out-of-Bag (OBB):** Samples excluded from a particular bootstrap subset (called out-of-bag samples) provide a natural validation set for that base model.

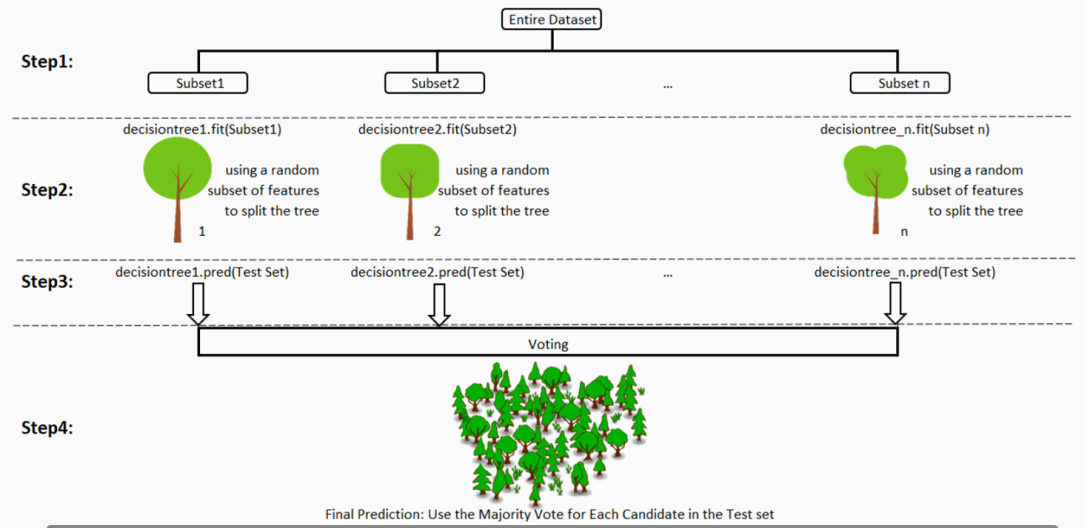
Random Forest Lec-18: Random Forest

Random forest is an ensemble classifier that consists of many **decision trees and outputs the class** that is the most voted of the class's output by individual trees.

Generally, **CART** trees are used with this method.

Steps:

1. Select n random subsets from the training set
2. Train n decision trees
    a. One **random subset** is used to train one decision tree
    b. The optimal splits for each decision tree are based on a **random subset of features**
3. Each individual tree predicts the records in the test set, independently
4. Make the final prediction
    a. For each candidate in the test set, Random Forest uses the class with hte majority vote as the candidates final prediction
    b. Mean for regression



Advantages

1. Most accurate algorithm for classifier
2. Runs on large database
3. Handle thousand of input variables, without variable deletion
4. Generates an internal unbiased estimate of the generalization
5. Effective method for estimating missing data and maintain accuracy
6. Methods for balancing error
7. Fast to build and predict

Disadvantages

- Overfit for some data with noisy classification/regression
- Random forest are biased in favor of the attributes with more level

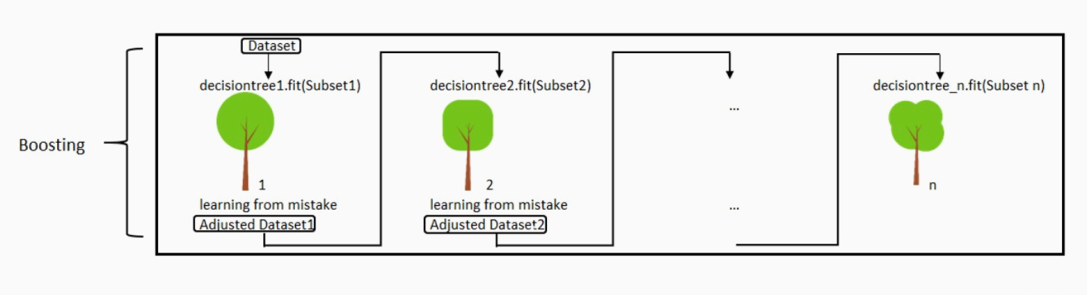
Boosting Boosting in Machine Learning 🔥

Boosting is ensemble learning method in machine learning, specially designed to improve the accuracy of predictive models by combine multiple weak learners (models that perform only slightly better than random guessing) into a single, strong learner.

Works

1. **Select initial weights:** Assign initial weights to all data points to indicate their importance in the learning process.

2. **Train Sequentially:** Train the **first weak** on the data. After evaluating its performance, **increase the weights of misclassified instances**. This makes the next weak learner focus more on the harder cases.
3. **Iterate Process:** Repeat the process of adjusting weights and training subsequent learners. Each new model focuses on the weaknesses of the ensemble thus for.

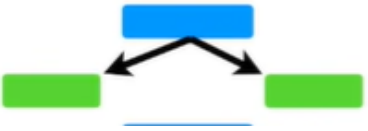


1. **Combine the Result:** Aggregate the **predictions of all weak learners** to form the final output. The aggregation is typically weighted, where more accurate learners have more influence.

Types of Boosting

AdaBoost What is AdaBoost (BOOSTING TECHNIQUES)

- **Adaboost (Adaptive Boosting)** combines a lot of “weak leaners” to make classifications. The weak learners are almost always stumps.
    - Stumps are a tree with one root and two leaves only.



        - So, this is really a Forest of Stumps rather than trees
- Some stumps get more say in the classification than others
- Each stump is made by taking the previous stumps mistake into accounting
    - Order matters

- The meta-learner adapts based upon the results of the weak classifiers
- Learns from the mistake increasing the weights of misclassified data points
- The ensemble model is defined as a weighted sum of weak learners

What can we boost

- Weak Learner produces hypotheses at least as good as random classifier
  - **Examples:** Rules of thumb, decision stumps, perceptron, Naive Bayes
  - **Advantages:**
    - No need to learn a perfect hypothesis
    - Can boost any any learning algorithm
    - Boosting is very simple to program
    - Good generalization

How does it work

1. Initialize the weights of data points.
2. Train a decision tree
3. Calculate the weighted error rate of the decision tree
4. Calculate this decision tree's wright in the ensemble

   $\alpha_t = \frac{1}{2}\ln(\frac{1-\epsilon_t}{\epsilon_t})$

   $\epsilon_t = \frac{\sum_{i=1}^{n} w_i l(y_i \neq h_t(x_i))}{\sum_{i=1}^{n} w_i}$
5. Update weights of wrongly classified points

   $\text{Others} = \text{Old Weight} \times e^{-\alpha}$

   $\text{Missclassified} = \text{Old Weight} \times e^{\alpha}$

   - Normalize the weights → summation of all weights should be 1
6. Repeat Step 1 (Until number of trees we set to train is reached)
7. Make the final predicction

The AdaBoost makes a new prediction by adding up the weights multiply the prediction,

$H(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$

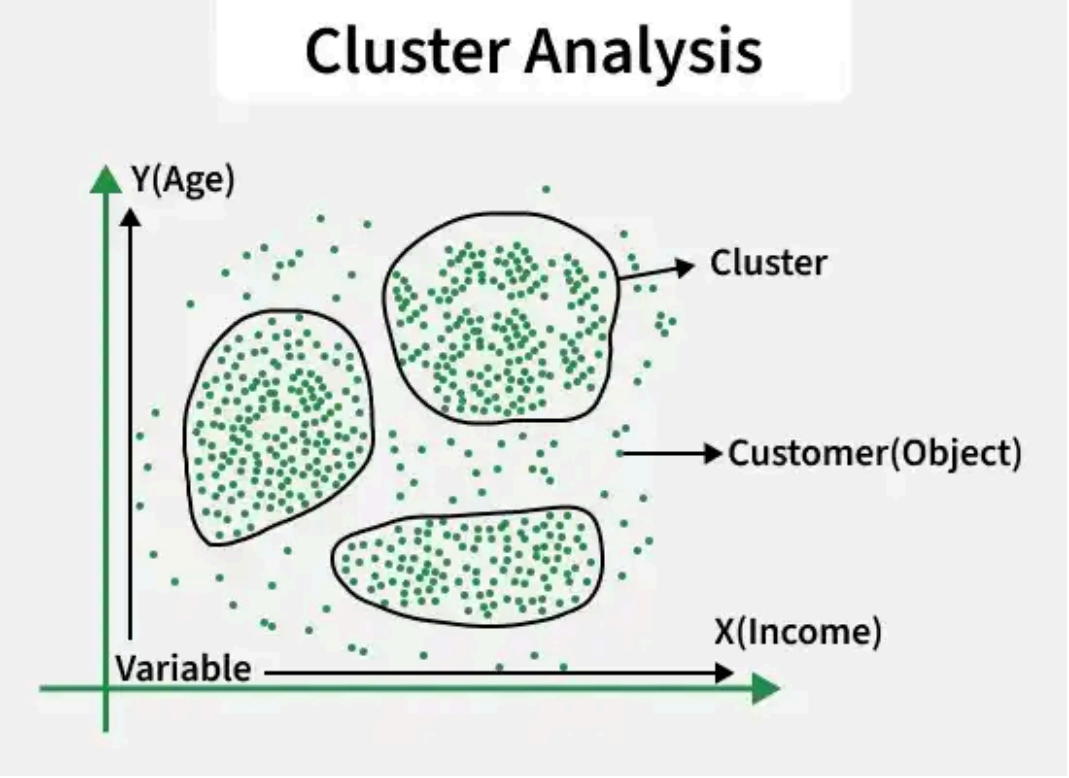
The tree with higher weight will have more power to influence the final decision.

Unsupervised Learning

Unsupervised learning is a type of machine learning where algorithms discover hidden patterns and structures in data without using labeled outputs or predefined categories.

Clustering

Given a dataset D with n tuples, $D = \{t_1, t_2, ..t_n\}$ and a parameter k, the Clustering Problem is to find mapping $f : D \Rightarrow \{1 \ldots, k\}$ where each t_i is assigned to one cluster k_j, $1 \leq j \leq k$



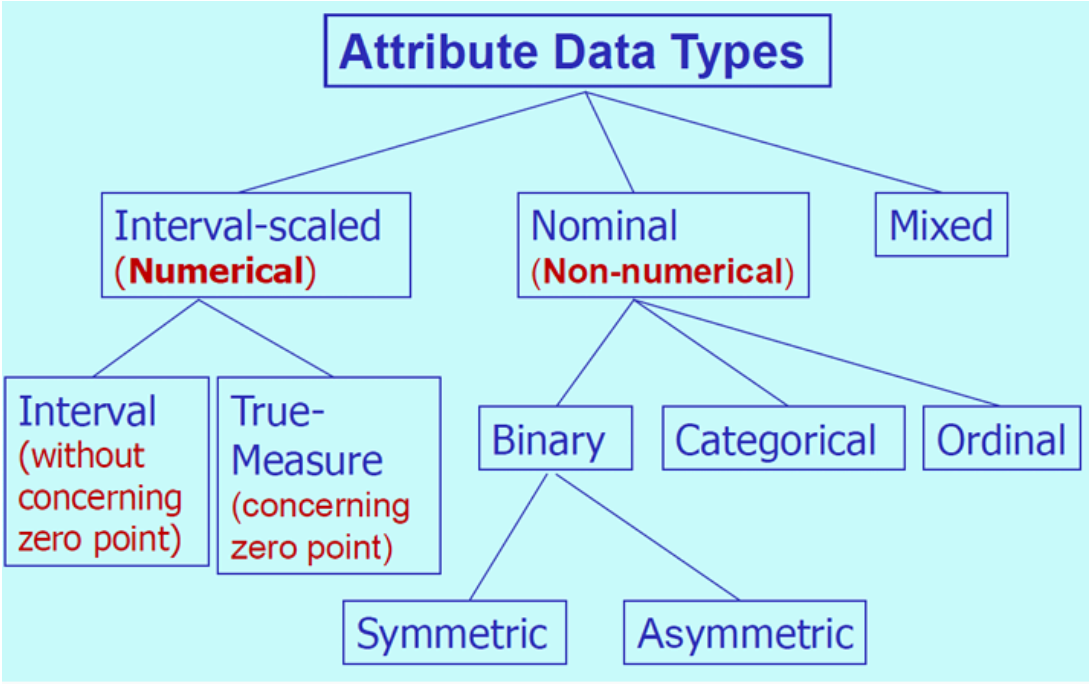
A cluster k_j contains precisely those **tuples mapped to it** (with the highest similarity).

A general similarity measure method : **Euclidean distance**

What is good Clustering

1. Produce high quality cluster
    a. High intra class similarity
    b. Low inter-class similarity
2. The quality of a clustering result depends on both the similarity measure used by the method and its implementation

Data types for Clustering/Main Attribute Types for Clustering



1. **Numerical**
    a. **Interval-Scaled Data :** The most common type of numerical data used in clustering. These are c**ontinuous measurement on a linear scale,** such as height, weight, temperature.
        i. There is no true zero point in interval data. Example: 0° doesn't mean the absence of temperature.

ii. Ratio has no meaning for interval variables. *Example:* $20^\circ C$ doesn't mean warmer twice than $10^\circ C$

b. **True Measure Variables/Ratio Scaled Data:**

i. Values have a true zero point ((zero represents the complete absence of the variable)

ii. ratios are meaningful (e.g., distance, age).

Example: A distance $1000km$ is twice far of a distance $500km$

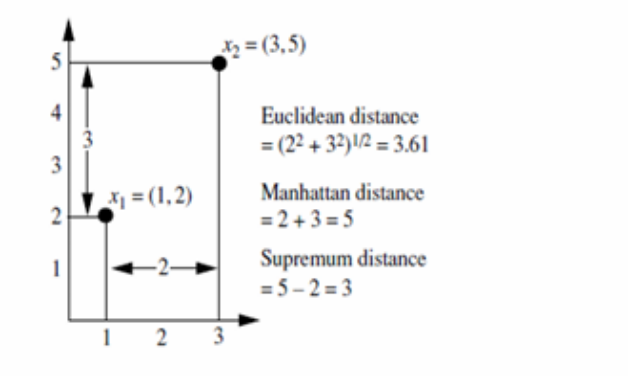
Dissimilarity in Numerical Data

Euclidean, $d(i,j) = \sqrt{(x_{i1} - x_{j1})^2 + \ldots + (x_{ip} - x_{jp})^2}$

Manhattan, $d(i,j) = |x_{i1} - x_{j1}| + \ldots + |x_{ip} - x_{jp}|$

Minkowski, $d(i,j) = \sqrt{|x_{i1} - x_{j1}|^h + \ldots + |x_{ip} - x_{jp}|^h}$, $h \geq 1$

Supremum distance, $d(i,j) = \max_f^p |x_{if} - x_{jf}|$



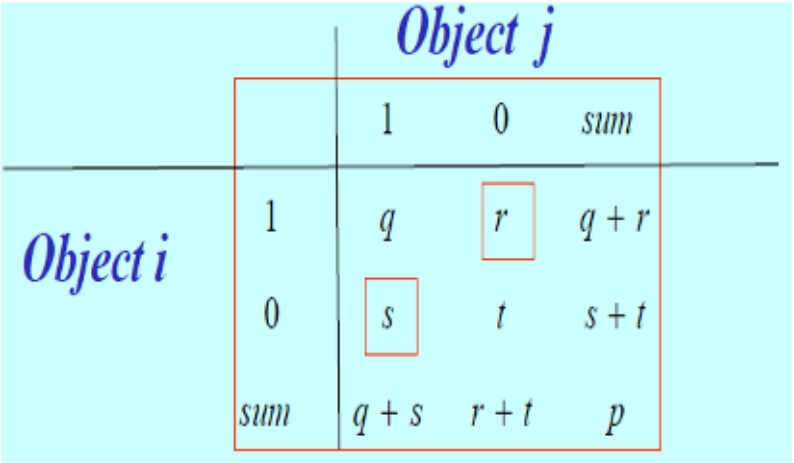
2. **Binary Variables**

a. Attributes with two states (0/1, yes/no, true/false)

b. Can be **symmetric** (both states equally important) or **asymmetric** (one state is significantly more important, e.g.: medical diagnosis)

c. **Dissimilarity and similarity** measures for objects described by either **symmetric or asymmetric**

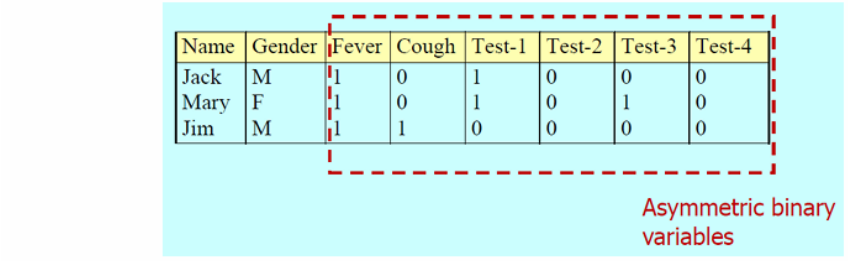
d. A contingency table



The total number of attributes $p = q + r + s + t$

- **Symmetric Binary Dissimilarity:** Dissimilarity that is based on symmetric binary attributes is called symmetric binary dissimilarity The dissimilarity between i and j, $d(i, j) = \frac{r+s}{q+r+s+t}$
- **Asymmetric binary dissimilarity:** The dissimilarity based on **Asymmetric** attributes is called asymmetric binary dissimilarity, where the number of negative matches t consider uninimportant.

  $d(i, j) = \frac{r+s}{q+r+s}$



Name	Gender	Fever	Cough	Test-1	Test-2	Test-3	Test-4
Jack	M	1	0	1	0	0	0
Mary	F	1	0	1	0	1	0
Jim	M	1	1	0	0	0	0

Asymmetric binary variables

- The distance between Jack and Jim, d(jack, jim) = 1+1/1+1+1 = 0.67
- The distance between Jack and Mary, d(jack, mary) = 0+1/2+0+1 = 0.33

- **Asymmetric binary similarity:** The similarity based on asymmetric attributes..

  $sim(i, j) = \frac{q}{q+r+s} = 1 - d(i, j)$, $sim(i, j)$ is called Jaccard coefficient.

3. **Categorical/Nominal Data**
   a. Categorical data represents the variable that can take on a limited **set of distinct non-numerical values,** such as color (red, green), brand type,

food etc. These valued have no inherent ordering.

Proximity Measures for Nominal Attributes: Consider only nominal attributes

The dissimilarity between two objects i and j can be computed based on the ratio of mismatch

$$d(i,j) = \frac{p-m}{p} = \frac{\text{mismatch}}{\text{total nominal attribute}};$$

$$m = \text{number of matches attribute}$$

Object Identifier	test-1 (nominal)	test-2 (ordinal)	test-3 (numeric)
1	code A	excellent	45
2	code B	fair	22
3	code C	good	64
4	code A	excellent	28

$$\begin{bmatrix} 0 & & & \\ 1 & 0 & & \\ 1 & 1 & 0 & \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Think about the $\text{row} = 3$,

$d(3,1) = 1 = \frac{1}{1}$, where,

Code A ≠ Code C, so, $m = 0$

$p - m = 1 - 0 = 1$

$p = 1 =$

there is only one nominal attri

4. **Ordinal Data**
   a. Ordinal data is similar to categorical data with a **meaningful order or ranking** among values. Example, Product rating (High, medium, low)
   b. The dissimilarity computation with respect to attribute f involves the following steps
      i. Replace each x_{if} by its corresponding rank, $r_{if} \in \{1, .., M_f\}$
      ii. Map the range of each attribute onto $[0.0, 1.0]$ so that each attribute has a equal wight (Normalize)
      $z_{if} = \frac{r_{if}-1}{M_f-1}$
      iii. Compute dissimilarity using a distance measure method

Object Identifier	test-1 (nominal)	test-2 (ordinal)
1	code A	excellent
2	code B	fair
3	code C	good
4	code A	excellent

	Order	Normalization
excellent	3	$\frac{3-1}{3-1} = 1$
good	2	$\frac{2-1}{3-1} = 0.5$
fair	1	$\frac{1-1}{3-1} = 0$

$$\begin{bmatrix} 0 & & & \\ 1.0 & 0 & & \\ 0.5 & 0.5 & 0 & \\ 0 & 1.0 & 0.5 & 0 \end{bmatrix}$$

5. **Mixed-type data:** Contain a mix of numerical, binary, categorical and ordinal attributes.
6. **Text-data or string data:**
    a. form of unstructured text, like production description
    b. Unstructured $\Rightarrow$ Structure
        i. TF-IDF (Term Frequency-Inverse Document Frequency)
        ii. Word embeddings (Word2Vec)
7. **Time-Series and Sequential data**
    a. consists of observations recorded over time, such as stock price, weather patter.

Data structure for clustering

The records in a **data set need to be mapped** into points multidimensional space.

With a dataset of n objects:

1. **Data Matrix (Object-by-variable structure):** A data matrix in clustering is the most common **way to represent your dataset before applying** clustering algorithm.
    a. **Two modes :** n objects by p variables
        i. Each cell contains the value of a particular attribute for a particular object.

b. A form of relational table or $n \times p$ matrix

c. It is used for serving **data preparation process.**

$$\begin{bmatrix} x_{11} & \cdots & x_{1f} & \cdots & x_{1p} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{i1} & \cdots & x_{if} & \cdots & x_{ip} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{n1} & \cdots & x_{nf} & \cdots & x_{np} \end{bmatrix}$$

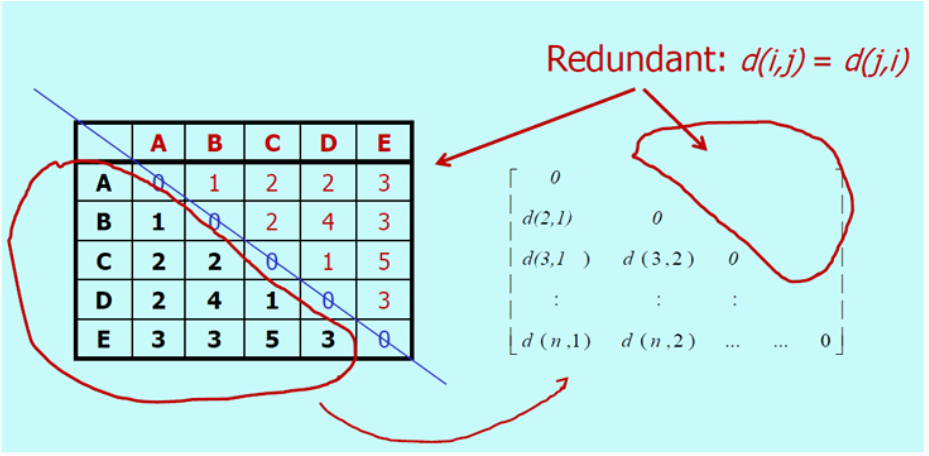
2. **Dissimilarity matrix (Object-by-object structure):** A dissimilarity matrix (also called an **object-by-object structure**) is another way to represent data for clustering. Instead of storing raw attributes, it stores the pairwise differences (distances or dissimilarities) between objects.

$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \cdots & \cdots & 0 \end{bmatrix}$$

1. **One mode:** n objects by n objects and

   a. $d(i,j) = d(j,i),$ the matrix is usually **symmetric**:

   b. $d(i,i) = 0$



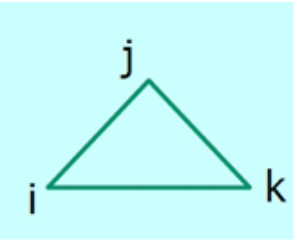
2. A form of $n \times n$ table, stores available differences for all pairs of n objects
3. Many clustering algorithms **operate on dissimilarity**

Dissimilarity Between Objects

Euclidean distance,

$$d(i,j) = \sqrt{|x_{i1} - x_{j1}|^2 + |x_{i2} - x_{j2}|^2 + ... + |x_{ip} - x_{jp}|^2}$$

Properties



- $d(i,j) \geq 0$
- $d(i,i) = 0$
- $d(i,j) = d(j,i)$
- $d(i,j) \leq d(i,k) + d(k,j)$

Basic Clustering Methods

Partitioning methods

- Partitioning method in clustering are a family of algorithms that divide a dataset into a set of **non-overlapping groups (cluster) directly, without building a hierarchy.**
- It is a single level clustering technique, data partition without hierarchy
- It creates clusters in one stage as opposed to several stages

- Find manually exclusive clusters of spherical shape
- Distance based
- Requires number of clusters k by user (Since only one set of clusters is output,)
- May use mean or medoid to represent cluster center
- Effective for small-to-medium-size data sets

Advantages	Limitations
Easy to implement	Must predefine k
Works well with large datasets	Sensitive to initialization
Fast convergence	Struggles with non-globular clusters
Useful for numeric data	Not ideal for mixed attribute types

K-means Clustering

K-means clustering is a partitioning method that divides a dataset into K **distinct, non-overlapping cluster,** where K is a number chosen by the user.

1. **Choosing k and initializing centroids:**
    a. Let training set, $T = x_1, \dots x_n$, each tuple represents a point in an m dimensional feature space
    b. Parameter K is used as the number of cluster.
        i. **Initialize centroids**: Randomly choose K objects in sequence from training set as the initial cluster centers, i.e $c_1, \dots, c_k$, represents k partitions.
            1. A **centroid** is the **center point of a cluster**. It represents the average position of all the data points belonging to that cluster.

               $$\text{Centroid} = \frac{1}{n} \sum x_i$$

            2. In **multi-dimensional data**, it's the average across all dimensions.
2. **Assign points**
    a. Each data point is assigned to the nearest centroid
    b. How ?

i. Determine the distance of each object to the centroid by using Euclidian distance
$dist(x_i, c_k) = \sqrt{\sum_s^m (x_{is} - c_{ks})^2}$
$k = 1..K, i = 1..n$

c. Group the objects based on minimum distance from cluster enter to each object and update cluster membership $C_1, \dots C_l$ based on the distance.

d. Grouping data set into K cluster to minimize the within cluster sum of square distance which is defined as
$\sum_k^K \sum_{j \in C_k} ||X_j - C_k||^2$

3. **Update centroids**

a. Recalculate the centroid of each cluster as the mean/averaging of all points in that cluster.
$c_k = \frac{1}{|c_k|} \sum_{j \to C_k} x_j$
$k = 1 \dots k$

4. Repeat the step 2 to step 3 until convergence.

Examples

1. **1D Dataset:** https://www.youtube.com/watch?v=5aBjP9Tn2Ic

❓

- Example: Given 1) a 1D data set: {2, 4, 10, 12, 3, 20, 30, 11, 25},
  2) K = 2.
- Methods:
  - Randomly assign initial cluster center: $c_1 = 3$, $c_2 = 4$
  - Calculate new centers, then relabel objects into K clusters until converge (i.e., new means/centers have no change):

1. K1={2, 3}, K2={4, 10, 12, 20, 30, 11, 25}, c_1=2.5, c_2=16
2. K1={2, 3, 4}, K2={10, 12, 20, 30, 11, 25}, c_1=3, c_2=18
3. K1={2, 3, 4, 10}, K2={12, 20, 30, 11, 25}, c_1=4.75, c_2=19.6
4. K1={2, 3, 4, 10, 11, 12}, K2={20, 30, 25}, c_1=7, c_2=25
5. **K1**={2, 3, 4, 10, 11, 12}, **K2**={20, 30, 25}, c_1=7, c_2=25 (No more change), iteration stops.

2. **2D Dataset :** https://www.youtube.com/watch?v=DfK-JmGlMyQ

❓ They differ only in the Euclidean distance formula.

Evaluating K-means cluster

1. **SSE : Sum of Squared Error (SSE)**
    a. **SSE** measures the compactness of clusters.
    b. It is the sum of squared distances between each data point and the centroid of its assigned cluster.

$$\text{SSE} = \sum_{i=1}^{k} \sum_{x \to C_i} ||x - m_i||^2$$

k = number of clusters

C_i = cluster i

m_i = centroid of cluster i

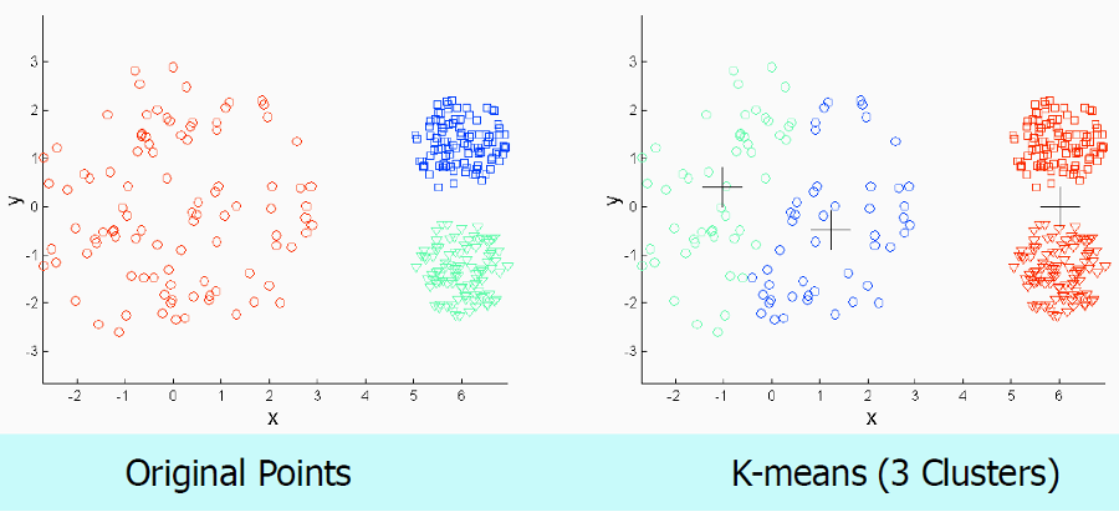
x = data point in cluster i

    c. **Lower SSE**
        a. Better Clustering
        b. Cluster are tighter and more compact
    d. **Higher SSE**
        a. Poorer Clustering
        b. Points are farther from their centroids
    e. **How to Reduce SSE**
        a. Increase k
            a. More cluster usually reduce SSE because points are closer to their centroid
            b. But too many clusters may lead to overfitting.
        b. Better initialization of centroids
        c. Remove outliers
        d. Feature scaling

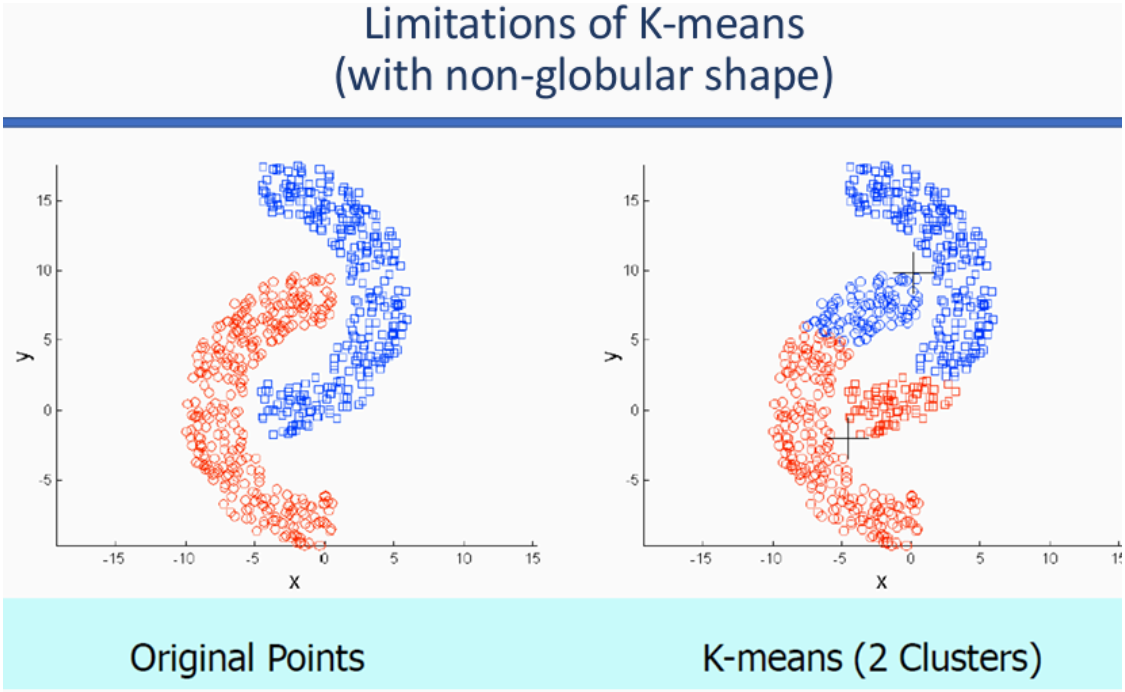
Properties of K-means

Limitation of K-means

- Selection of k and initial cluster centers.
- Unable to handle datasets with different types of attributes.



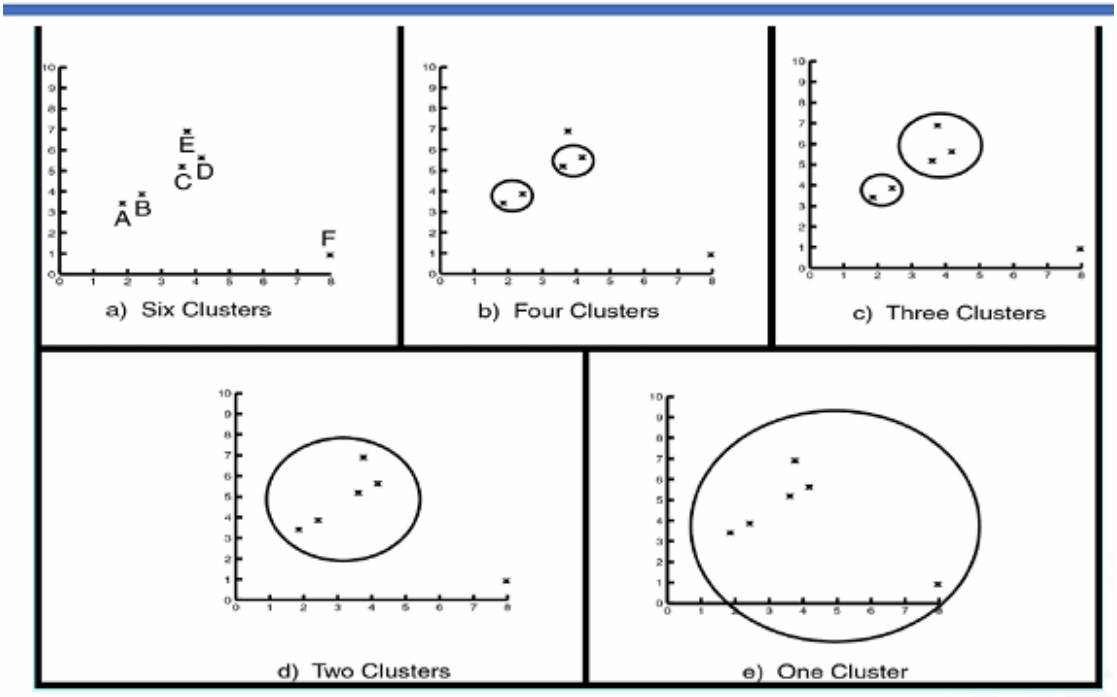
- Final solution after convergence might not be the global minimum.
- Does not work well with non-globular clusters.



Impact of Outliers on Clustering

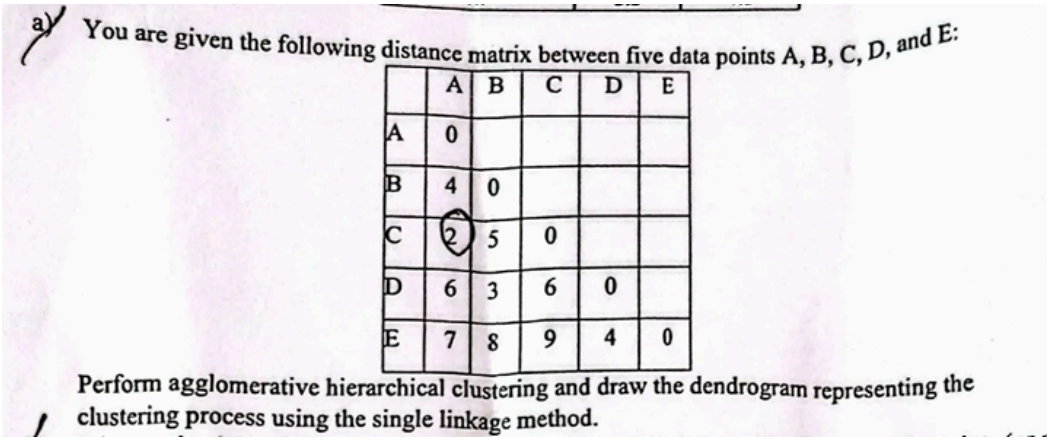
Hierarchical Clustering

- Hierarchical clustering builds a **tree-like structure (dendrogram)** of cluster. Instead of partitioning data into a fixed numbers of cluster directly, it creates a hierarchy of cluster that can be cut at different levels to form groups.



- Approach: https://www.youtube.com/watch?v=zxQF8Rmpk1M
  - **Agglomerative (Bottom up)**
    - Start with each data point as its cluster
    - Iteratively merge the two closet clusters until only one cluster remains
    - Most common approach
  - **Divisive (Top-Down)**
    - Start with all data points in one cluster.
    - Iteratively split clusters into smaller ones until each point is alone.
    - Less common, but useful in some contexts.
- **How clusters are merged (Linkage Criteria)**

- **Single Linkage:** Distance between the closet points of two clusters https://www.youtube.com/watch?v=pbTQQCA9Xs0



a) You are given the following distance matrix between five data points A, B, C, D, and E:

	A	B	C	D	E
A	0				
B	4	0			
C	2	5	0		
D	6	3	6	0	
E	7	8	9	4	0

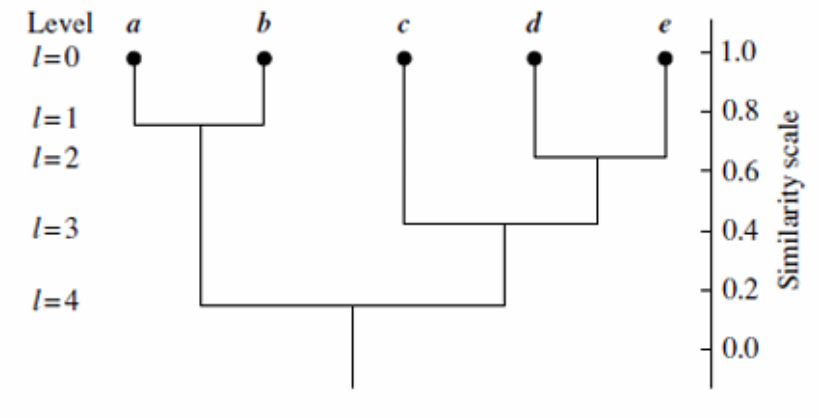
Perform agglomerative hierarchical clustering and draw the dendrogram representing the clustering process using the single linkage method.

Agglomerative Hierarchical Clustering (Single Link)

Step	Minimum Distance	Merge	Clusters After Merge
0	–	–	{A}, {B}, {C}, {D}, {E}
1	2	A + C	{AC}, {B}, {D}, {E}
2	3	B + D	{AC}, {BD}, {E}
3	4 *(Tie: AC–BD and BD–E)*	AC + BD *(First minimum encountered)*	{ABCD}, {E}
4	4	ABCD + E	{ABCDE}

- **Complete Linkage:** Distance between the farthest points of two clusters.
- **Average Linkage:** Average distance between all pairs of points across cluster.
- **Ward's method:** Merge cluster that result in the smallest increase in total variance.

- **Dendrogram**
  - A dendrogram is a tree diagram showing the **sequence of merges or splits.**
  - Each level shows clusters for that level.
    - Leaf - individual clusters
    - Root - one cluster



- By “cutting” the dendrogram at a certain height, you decide how many clusters to form.

Advantages	Limitations
No need to predefine number of clusters	Computationally expensive for large datasets
Produces a hierarchy (multi-level clustering)	Sensitive to noise and outliers
Visual representation via dendrogram	Choice of linkage affects results
Can capture nested clusters	Not scalable for very large datasets

- **Density Based Clustering**
  - Density-based methods group data points into cluster based on regions of high point density. Instead of forcing every point into a cluster, they can identify **noise/outliers** naturally.
  - Can find arbitrary shaped clusters
  - Clusters are dense regions of objects in space that are separated by low-density regions
  - Cluster density: Each point must have minimum number of points within its neighbhorhood
  - May filter out outliers
  - **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**
    - A cluster is formed when there are enough points within a given radius ϵ
    - **Parameters**

- **ε (epsilon)**: Radius of neighborhood around a point.
- **MinPts**: Minimum number of points required to form a dense region.

Advantages	Limitations
Finds clusters of arbitrary shape	Struggles with varying densities
Automatically detects noise/outliers	Sensitive to choice of ε and MinPts
No need to specify number of clusters	Computationally expensive for very large datasets
Works well for spatial/geographical data	Parameter tuning can be tricky

- **Grid-based methods**
  - Grid-based methods divide the data space into a **finite number of cells (grids)** and then **perform clustering on the grid structure** rather than directly on the data points.
  - Use a multiresolution grid data structure
  - Fast processing time
  - **How it works**
    - **Divide the space**: The data space is partitioned into a grid of cells.
    - **Count density**: Each cell is assigned a density value based on the number of points inside it.
    - **Cluster formation**: Dense cells are grouped together to form clusters, while sparse cells are treated as noise.
    - **Output**: Clusters are defined by connected dense regions in the grid.

Advantages	Limitations
Fast and efficient for large datasets	Grid resolution affects results
Can detect clusters of arbitrary shape	May struggle with very sparse data
Works well with spatial data	Sensitive to choice of grid size
Handles noise effectively	Less flexible than density-based methods

Deep learning

Deep learning is a branch of **machine learning** that uses artificial neural networks with many layers ("deep" networks) to learn patterns from large amounts of data.

A deep learning model learns in three basic steps:

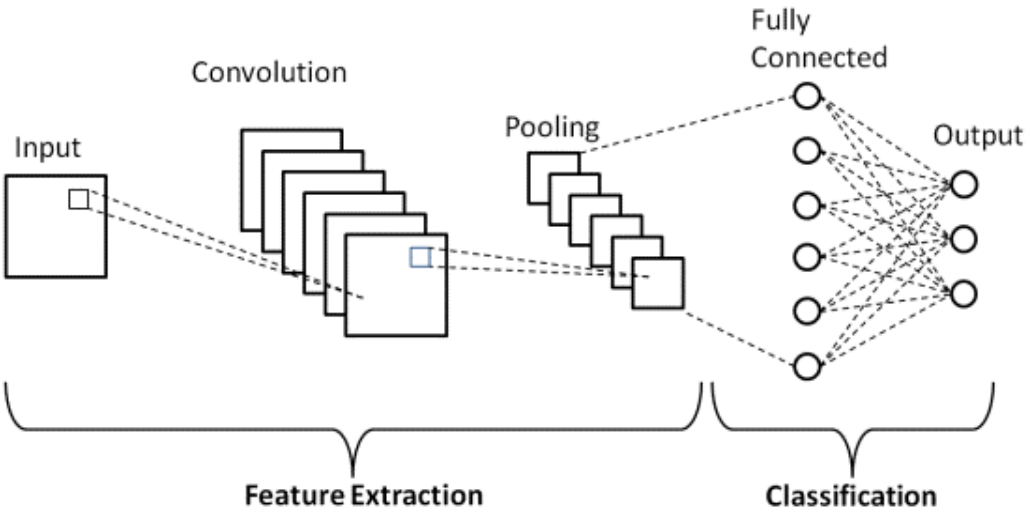
1. **Input:** It receives data (such as images, text, or audio).
2. **Hidden layers:** Multiple layers process the data, extracting increasingly complex features.
3. **Output:** It produces a prediction or decision.

CNN https://www.youtube.com/watch?v=HGwBXDKFk9I

A **Convolutional Neural Network (CNN)** is a specialized type of **Artificial Neural Network (ANN)** designed primarily for processing structured grid-like data such as images. CNNs automatically learn important features (edges, textures, shapes, and objects) from input images, eliminating the need for manual feature extraction.

CNNs use **convolution operations** to efficiently process images while significantly reducing the number of trainable parameters.

CNN Architecture



Basic CNN architecture

1. **Input Layer:** The input layer receives the raw image as a multidimensional array of pixel values.

   For an RGB image:

   $$\text{Input Size} = H \times W \times C$$

   where,

   $H =$ Height

   $W =$ Weight

   $C =$ Number of channels

2. **Convolutional Layer:**
   a. Core component of CNN
   b. It extracts local features from the input by applying learnable filters/kernels through convolution operation to produce feature maps.

      $$S(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

      where,

      $I =$ Input image

      $K =$ Kernel

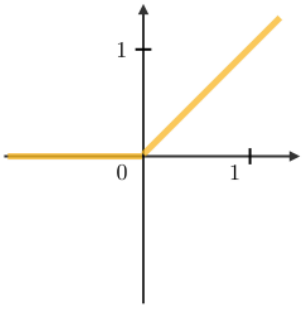
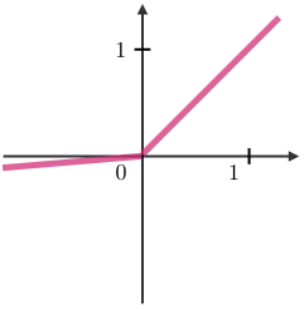
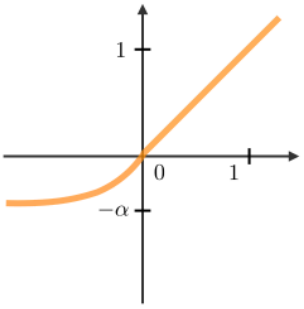
      $S =$ Output Feature map

   c. **Purpose**
      a. Detect edges
      b. Detect textures
      c. Learn patterns automatically

3. **Activation Layer:**
   a. The activation layer introduces non-linearity into the network, enabling it to learn complex relationships.
   b. The most commonly used activation function is **Rectified Linear Unit (ReLU).**
   c. **Purpose**
      i. Introduces non-linearity

ii. Reduces vanishing gradient problem

iii. Improves training speed

ReLU	Leaky ReLU	ELU
$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$	$g(z) = \max(\alpha(e^z - 1), z)$ with $\alpha \ll 1$
		
• Non-linearity complexities biologically interpretable	• Addresses dying ReLU issue for negative values	• Differentiable everywhere

https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/

4. **Pooling Layer**

   a. The pooling layer **reduces** the spatial dimensions of feature maps while retaining the most important information.

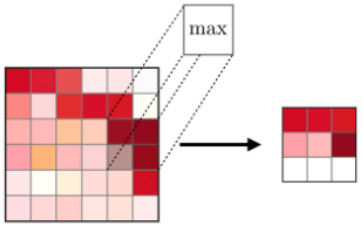
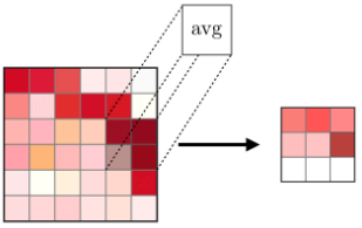
   b. The pooling layer (POOL) is a **downsampling** operation, typically applied after a convolution layer, which does some spatial invariance

   c. **Purpose**

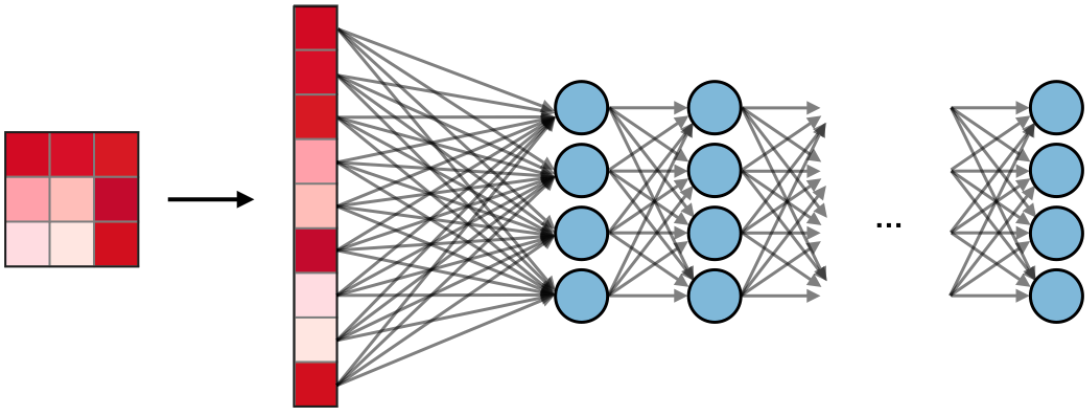
      i. Reduces computation

      ii. Reduces overfitting

      iii. Preserves dominant features

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

5. **Flatten Layer**
    a. The flatten layer converts the **multidimensional feature maps into a one-dimensional vector** that can be processed by fully connected layers.
    b. **Purpose:** Connect convolutional layers with dense layers.
6. **Fully Connected Layer**
    a. A fully connected layer connects every neuron to all neurons in the previous layer and performs high-level reasoning for classification.
    b. The fully connected layer (FC) operates on a flattened input where each input is connected to all neurons. If present, FC layers are usually found towards the end of CNN architectures and can be used to optimize objectives such as class scores.



1. The output is computer as

$$z = W^T x + b$$

Where,
$W =$ Weight matrix

$x =$ input vector

$b =$ bias

7. **Output Layer**
    a. The output layer produces the final prediction of the network.
    b. **Common Activation functions**
        i. **Sigmoid** for binary classification
        $$\sigma(x) = \frac{1}{1 + e^{-x}}$$
        ii. Softmax for multiclass classification
        $$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Functionality of CNN

A CNN performs image recognition by automatically learning hierarchical features from input images. During training, convolution layers extract features, activation functions introduce non-linearity, pooling layers reduce dimensionality, and fully connected layers classify the extracted features. The network parameters are optimized using backpropagation and gradient descent.

Advantages

- Automatically extracts features from images.
- Reduces the number of trainable parameters through weight sharing.
- Preserves spatial information.
- Achieves high accuracy in computer vision tasks.
- Reduces the need for manual feature engineering.

Applications

- **Image classification**

- Image classification is the task of assigning a predefined class label to an entire image. A CNN extracts relevant features from the image and predicts the class with the highest probability.
- **Examples:**
  - Cat vs. dog classification
  - Handwritten digit recognition (MNIST)
  - Plant disease identification
  - Medical image diagnosis

Output: One label per image.

- **Video Classification**
  - Video classification is the process of assigning a category or action label to an entire video sequence. CNNs extract spatial features from individual frames, often combined with temporal models to capture motion information.
  - **Examples:**
    - Human activity recognition
    - Sports event classification
    - Violence detection
    - Video recommendation systems

Output: One label per video.

- **Object detection**
  - Object detection is the task of identifying, locating, and classifying one or more objects within an image. CNNs predict both the object class and its bounding box coordinates.
  - Example
    - Face detection
    - Pedestrian detection
    - Vehicle detection
    - Traffic sign recognition
    - Security surveillance

    - **Output:** Object class and bounding box for each detected object.
- **Semantic Segmentation**
    - Semantic segmentation is the process of classifying every pixel in an image into a specific category. Unlike image classification, it provides pixel-level understanding of the scene.
    - **Examples:**
        - Road and lane detection
        - Medical image segmentation (tumor detection)
        - Satellite image analysis
        - Land cover mapping
    - **Output:** A pixel-wise labeled image.
- **Face recognition**
    - CNNs are used to identify or verify a person's identity by learning distinctive facial features.
- **Medical image analysis**
    - CNNs assist in analyzing medical images to detect diseases and abnormalities with high accuracy.
        - Tumor detection
        - Pneumonia detection from chest X-rays
        - MRI and CT scan analysis
        - Retinal disease detection
- **Autonomous vehicles**
    - CNNs enable self-driving vehicles to understand their surroundings by detecting roads, lanes, pedestrians, traffic signs, and other vehicles.
        - Lane detection
        - Obstacle detection
        - Traffic sign recognition
- **Handwritten digit recognition**
    - CNNs recognize and extract text from images or scanned documents.

    - Document digitization
    - Automatic number plate recognition
    - Handwritten text recognition
- **Video surveillance**

Application	Purpose	Output
Image Classification	Classify an entire image	Single class label
Video Classification	Classify an entire video	Single action or event label
Object Detection	Detect and locate multiple objects	Class labels with bounding boxes
Semantic Segmentation	Classify every pixel in an image	Pixel-wise labeled image
Face Recognition	Identify or verify a person's identity	Identity label
Medical Image Analysis	Detect diseases from medical images	Disease classification or localization
Autonomous Driving	Understand road scenes	Detected objects, lanes, and traffic signs
OCR	Recognize text from images	Extracted text
Emotion Recognition	Identify facial expressions	Emotion label